

实验一 开发环境搭建

一、实验目的

1. 掌握 .NET MAUI 跨平台开发环境的搭建流程
2. 学习使用 Visual Studio 2026 创建和管理 C# 项目
3. 实现登录功能和页面跳转（含前后端分离架构）
4. 了解 Git 版本控制的基本操作
5. 掌握项目部署和运行的完整流程

二、实验环境

- 操作系统：Windows 10/11
- 开发工具：Visual Studio 2026
- 框架：.NET 10.0 (MAUI + ASP.NET Core Minimal API)
- 前端：MAUI XAML
- 数据库：MySQL (Pomelo.EntityFrameworkCore.MySql)
- 版本控制：Git
- 远程仓库：Gitee

实验步骤

1. 环境搭建

1. 安装 Visual Studio 2026
 - 下载并安装 Visual Studio 2026
 - 勾选 ".NET MAUI" 工作负载
2. 创建项目
 - 使用命令 `dotnet new webapp -o SGTPDILoginApp` 创建 ASP.NET Core Web 应用
 - 进入项目目录：`cd SGTPDILoginApp`
3. Git 初始化
 - 初始化 Git 仓库：`git init`
 - 连接远程仓库：`git remote add origin https://gitee.com/chzucz1d1/cg2-sgtpdi.git`
 - 创建并切换分支：`git checkout -b SGTPDI-C#+.NET+MAUI`

2. 功能实现

1. 登录页面
 - 创建 LoginPage.xaml 页面，实现登录表单（账号 + 密码输入框 + 登录按钮）
 - 实现 `AuthService.LoginAsync()` 中的登录验证逻辑：对密码进行 SHA256 哈希后发送至后端 API 验证
 - 账号：2023210643，密码：123456

2. 学生信息页面

- 创建页面，显示学生信息
- 实现退出登录逻辑

3. 页面样式

- 使用 MAUI XAML 实现现代化页面样式
- 去掉默认的 bar 栏和 footer 栏
- 实现现代化的登录界面和学生信息页面

3. 项目部署

1. 构建项目

- 运行命令：`dotnet build`

2. 运行项目

- 运行命令：`dotnet run`

3. 代码提交

- 添加文件到暂存区：`git add .`
- 提交代码：`git commit -m "feat: 实现登录功能和学生信息展示"`
- 推送代码：`git push origin SGTPDI-C#+.NET+MAUI`

实验结果

1. 登录功能

- **登录页面：**显示二手交易平台登录表单，包含账号和密码输入框
- **登录验证：**输入正确的账号和密码后，跳转到学生信息页面
- **错误提示：**输入错误的账号或密码时，显示错误提示信息
- **对应截图：**

校园二手交易平台

学生



管理员

学号

请输入学号

密码

请输入密码



记住密码

登录



我的



关于

核心代码：登录验证逻辑

客户端 (AuthService.cs) —— 密码哈希与API请求：

```
public class AuthService
{
    public static readonly HttpClient Http = new() { Timeout =
    TimeSpan.FromSeconds(15) };

    // 密码 SHA256 哈希，避免明文传输
    private static string HashPassword(string password)
    {
        using var sha = System.Security.Cryptography.SHA256.Create();
        var bytes = System.Text.Encoding.UTF8.GetBytes(password);
        return Convert.ToHexString(sha.ComputeHash(bytes)).ToLower();
    }

    public async Task<(bool Success, string Message, StudentInfo? Info)>
    LoginAsync(
        string studentId, string password)
    {
        try
        {
            var payload = JsonSerializer.Serialize(
                new { student_id = studentId, password = HashPassword(password)
            });

            var content = new StringContent(payload, Encoding.UTF8,
            "application/json");
            var resp = await Http.PostAsync($"{ApiBase}/api/login", content);
            var json = await resp.Content.ReadAsStringAsync();

            var doc = JsonDocument.Parse(json).RootElement;
            if (!doc.TryGetProperty("success", out var successProp))
                return (false, "服务器错误", null);
            if (!successProp.GetBoolean())
                return (false, doc.GetProperty("message").GetString() ?? "登录失
            败", null);

            var data = doc.GetProperty("data");
            var info = new StudentInfo
            {
                StudentId = data.GetProperty("studentId").GetString() ?? "",
                Username = data.GetProperty("name").GetString() ?? "",
                IsAdmin = data.GetProperty("isAdmin").GetInt32() == 1
            };
            return (true, "登录成功", info);
        }
        catch (HttpRequestException ex)
        {
            return (false, $"无法连接服务器: {ex.Message}", null); }
    }
}
```

```

    }
}

```

服务端 (Program.cs) —— 用户查询与密码校验:

```

app.MapPost("/api/login", async (LoginRequest req, AppDbContext db) =>
{
    var user = await db.Users.FirstOrDefaultAsync(u => u.StudentId ==
req.student_id);
    if (user == null)
        return Results.Ok(new { success = false, message = "用户不存在" });
    if (user.Password != req.password) // 密码已在前端 SHA256 哈希
        return Results.Ok(new { success = false, message = "密码错误" });

    return Results.Ok(new {
        success = true,
        message = "登录成功",
        data = new {
            studentId = user.StudentId ?? "",
            name      = user.Username ?? "",
            avatar    = user.Avatar ?? "",
            isAdmin   = user.IsAdmin
        }
    });
});
});

```

客户端 (LoginPage.xaml.cs) —— 登录按钮事件处理:

```

private async void OnLoginClicked(object sender, EventArgs e)
{
    string id = StudentIdEntry.Text?.Trim() ?? "";
    string pwd = PasswordEntry.Text ?? "";

    if (string.IsNullOrEmpty(id) || string.IsNullOrEmpty(pwd))
    {
        _toastService.Show("请输入账号和密码");
        return;
    }

    var (success, message, info) = await _auth.LoginAsync(id, pwd);

    if (!success || info == null)
    {
        _toastService.Show(message);
        return;
    }

    AppConfig.SaveSession(info); // 保存登录状态
    App.NavigateToMain(info);    // 跳转主页面
}

```

2. 学生信息页面

- **信息展示：**显示学生学号（2023210643）和姓名（张海天）等个人信息
- **退出登录：**点击退出登录按钮，返回登录页面
- **对应截图：**



张海天

2023210643

学籍信息

学号 2023210643

姓名 张海天

学院 计算机与信息工程学院

专业 软件工程

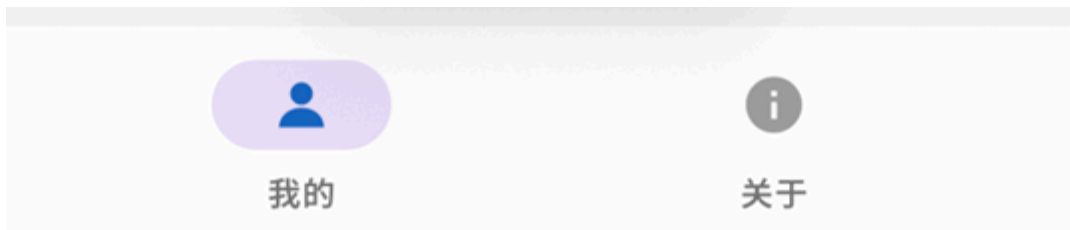
班级 软件232

修改密码

退出登录



欢迎，张海天



3. 页面样式

- **现代化设计**：实现现代化的页面样式
- **响应式布局**：适配不同屏幕尺寸
- **交互效果**：添加按钮悬停效果和输入框焦点效果

实验总结

本次实验成功完成了 .NET MAUI 开发环境的搭建和二手交易平台的开发。通过实验，我掌握了以下内容：

1. **环境搭建**：学会了如何安装 Visual Studio 2026 和 .NET 10.0，以及如何创建 MAUI 跨平台应用。
2. **功能实现**：实现了登录功能和学生信息展示功能，包括账号密码验证、页面跳转和退出登录逻辑。
3. **页面设计**：实现了现代化的页面样式，提高了用户体验。
4. **版本控制**：学习了 Git 的基本操作，包括初始化仓库、创建分支、提交代码和推送代码。
5. **项目部署**：掌握了项目的构建和运行流程，能够将项目部署到本地服务器。

通过本次实验，我对 C# 和 .NET MAUI 开发有了更深入的了解，为后续的项目开发打下了坚实的基础。

遇到的问题及解决方案

问题一：数据库字段为NULL导致类型转换异常

问题描述：登录时部分账号返回 `Unable to cast object of type 'System.DBNull' to type 'System.String'` 错误。

原因分析：数据库 `users` 表中部分字段（如 `avatar`、`qq`、`username` 等）允许为 `NULL`，但 EF Core 实体类中将这些字段定义为非空 `string` 类型。当数据库中该字段值为 `NULL` 时，`DBNull` 无法隐式转换为 `string`，导致运行时异常。

解决方案：

- 将实体类中所有可能为 `NULL` 的字段类型从 `string` 改为 `string?`（可空引用类型）
- 在 API 返回数据时使用空值合并运算符 `?? ""`，确保返回空字符串而非 `null`
- 在客户端解析 JSON 时使用 `TryGetProperty + ?? ""` 的安全访问模式，避免字段缺失时崩溃

问题二：数据库表结构与实体类字段不匹配

问题描述：创建交易时后端报错 `An error occurred while saving the entity changes`。

原因分析：代码中的 `Transaction` 实体类新增了 `spec_id`、`spec_name`、`quantity` 等字段，但数据库 `transactions` 表尚未执行对应的 `ALTER TABLE` 添加这些列，导致 EF Core 在 `SaveChanges` 时无法写入这些字段。

解决方案：编写并执行 SQL 迁移脚本，为 `transactions` 表添加缺失的列：

```
ALTER TABLE `transactions` ADD COLUMN `spec_id` int(11) NOT NULL DEFAULT 0 AFTER `product_name`;  
ALTER TABLE `transactions` ADD COLUMN `spec_name` varchar(100) AFTER `spec_id`;
```

问题三：HTTP请求流意外中断

问题描述：购买商品时出现 `unexpected end of stream on com.android.okhttp.Address@...` 网络错误。

原因分析：客户端发送的购买请求JSON中缺少 `quantity` 字段（后端新增了该参数但客户端未同步更新），导致后端反序列化失败，连接被异常关闭。

解决方案：在客户端 `ProductService.CreateTransactionWithPurchaseAsync()` 的 payload 中添加 `quantity` 字段，确保与后端 `TransactionCreateRequest` 的参数一致。

问题四：登录密码验证的安全隐患

问题描述：当前方案中，前端将密码进行SHA256哈希后传输，后端直接将收到的哈希值与数据库中存储的哈希值进行比对。这种方式虽然避免了明文传输，但存在“哈希传递攻击”（Pass-the-Hash）的风险——如果攻击者截获了哈希值，可以直接用该哈希值伪造登录请求，哈希值实际上变成了另一种“明文”。

改进思路：

- **挑战-应答方案：**客户端请求登录时，服务端先生成一个随机挑战码（nonce）返回给客户端，客户端将 `SHA256(password + nonce)` 作为应答发送，服务端用数据库中存储的密码哈希与相同的nonce计算后比对。由于每次nonce不同，即使截获应答也无法重放。
- **加盐哈希方案：**为每个用户生成唯一的盐值（salt），数据库存储 `SHA256(password + salt)`，即使两个用户密码相同，存储的哈希值也不同，有效防止彩虹表攻击。
- **HTTPS 传输加密：**在生产环境中启用HTTPS/TLS，确保整个通信通道加密，从根本上防止中间人截获数据。

以上安全改进方案将在后续版本中逐步实施。

AI辅助编码

在本次实验中，我使用了 TRAE AI 编码助手辅助开发，以下是具体的AI辅助过程：

1. 登录验证逻辑的AI辅助生成

提示词：“请帮我实现一个登录验证方法，要求：1) 密码使用SHA256哈希后传输，不传明文；2) 向后端 `/api/login` 发送POST请求；3) 解析返回的JSON，提取用户信息；4) 处理各种异常情况（超时、网络错误、非JSON响应）”

AI生成的代码：AI生成了 `AuthService.LoginAsync()` 方法的完整实现，包括 `HashPassword()` 辅助方法。生成的代码使用了 `System.Text.Json` 进行JSON序列化/反序列化，使用 `HttpClient.PostAsync()` 发送请求，并对返回结果进行了分层校验（先检查 `success` 字段，再提取 `data`）。

我的修改：

- 调整了API地址为动态读取 `AppConfig.ApiBase`，而非硬编码
- 增加了对空响应和非JSON响应的检查逻辑（如代理/网关错误返回HTML的情况）
- 将 `StudentInfo` 属性映射从硬编码改为安全的 `TryGetProperty` 方式，避免字段缺失时崩溃

2. 后端登录API的AI辅助生成

提示词："使用ASP.NET Core Minimal API实现 /api/login 接口，接收student_id和password，查询MySQL数据库验证，返回JSON格式的登录结果"

AI生成的代码：AI生成了完整的 `MapPost("/api/login", ...)` 端点，包括数据库查询、密码比对和JSON响应。

我的修改：

- 添加了 `try-catch` 异常处理，避免服务端未处理异常导致500错误
- 对所有可能为NULL的数据库字段添加了 `?? ""` 空值合并处理，解决 `DBNull` 类型转换异常
- 增加了 `/api/auth/login` 别名端点，兼容不同客户端调用

3. 登录页面UI的AI辅助

提示词："用MAUI XAML创建一个现代化的登录页面，包含账号输入框、密码输入框、登录按钮，风格简洁大方"

AI生成的代码：AI生成了 `LoginPage.xaml` 的基础布局，使用了 `Border` + `Shadow` 实现卡片式设计。

我的修改：

- 调整了配色方案，统一为蓝色主题
- 添加了"记住密码"复选框和历史账号下拉功能
- 修改了"学号"标签为"账号"，去除了"校园"相关字眼